

RouteSmith: Contextual Bandit Routing for Large Language Models

Yunpeng Liu-Lupo

RouteSmith Contributors

<https://github.com/yunpengl9071/routesmith>

June 2026

Abstract

Problem. Deploying large language models in production requires routing each query to the right model—frontier models deliver superior quality but cost 10–80× more than smaller alternatives. Existing routers (RouteLLM, Not Diamond) require 55K+ pretraining labels, are architecturally limited to binary strong/weak choices, and cannot adapt online to shifting query distributions. **Approach.** We frame LLM routing as a contextual bandit problem and present CB-RouteSmith, which applies LinUCB and Linear Thompson Sampling (LinTS) over a 27-dimensional query feature space. Both algorithms require zero pretraining labels, learn online from production feedback, scale to $K > 2$ arms without architectural changes, and add sub-millisecond routing latency.

Results. On binary routing (GPT-4o vs. GPT-4o-mini, 600 MMLU + 300 GSM8K questions with real API calls), LinTS-27d achieves APGR=0.593 with 46% cost reduction versus Always-Strong, while LinUCB-27d achieves APGR=1.126 by selective strong-arm routing. On 5-arm multi-model routing (GPT-4o, Claude-Sonnet-4.5, Qwen-Plus, MiniMax-M1, DeepSeek-V3)—structurally impossible for binary routers—LinTS achieves 71.0% accuracy with 45% cost savings, converging to a stable policy across seeds. Online bandits learn from ~100 queries, compared to 55K+ labels required by supervised approaches.

Conclusion. Contextual bandits provide a practical, zero-shot alternative to supervised routing, especially for multi-model deployments where pretraining labels are unavailable. The full RouteSmith system additionally includes a random forest predictor and semantic caching for production scenarios with offline calibration data.

1 Introduction

Large language models (LLMs) have become essential infrastructure, but deployment introduces a fundamental cost-quality tension: frontier models like GPT-4o deliver superior performance yet cost 16× more per token than smaller models like GPT-4o-mini. For applications processing millions of queries, selecting the right model for each query is critical to managing costs while maintaining quality [Ong et al. \[2024\]](#), [Chen et al. \[2023\]](#), [Shnitzer et al. \[2023\]](#).

Existing routing approaches fall into two categories. Static supervised routers (RouteLLM [Ong et al. \[2024\]](#), Not Diamond [Contributors \[2024\]](#), FrugalGPT [Chen et al. \[2023\]](#)) learn classifiers from large human-preference datasets (55K+ labels for RouteLLM-SW) and apply them at inference time without adaptation. Heuristic cascades (FrugalGPT [Chen et al. \[2023\]](#), AutoMix [Aggarwal et al. \[2024\]](#), Dekoninck et al. [Dekoninck et al. \[2025\]](#)) call a weak model first and escalate based on confidence, but require threshold tuning and cannot handle simultaneous multi-model selection.

These approaches share three structural limitations: (1) they require substantial pretraining data before routing any query, (2) they are architecturally binary or require $O(K^2)$ separate classifiers for K models, and (3) they do not adapt to changing query distributions or model behavior in production.

Our approach. We frame LLM routing as a contextual bandit problem. At each timestep, a query arrives with context features, the router selects a model (arm), and observes a reward reflecting response quality minus cost. Unlike full RL, routing decisions across queries are independent (no state transition), making the contextual bandit formulation both more appropriate and more sample-efficient.

We present CB-RouteSmith, which applies two contextual bandit algorithms over a 27-dimensional query feature space:

- LinUCB-27d: Maintains per-arm linear ridge regression models with UCB exploration, achieving strong APGR by combining principled exploration with per-query linguistic features.
- LinTS-27d: Applies Linear Thompson Sampling [Abeillé and Lazaric \[2017\]](#) to LLM routing, eliminating the β exploration hyperparameter via posterior sampling.

Contributions:

1. Contextual bandit routing for LLMs: Application of LinUCB and LinTS [Li et al. \[2010\]](#), [Abeillé and Lazaric \[2017\]](#) to LLM routing over a 27-dim linguistic feature space. LinUCB-27d achieves APGR=1.126 on MMLU; LinTS-27d (parameter-free) achieves APGR=0.593 with 46% cost reduction.
2. Binary routing evaluation (Experiment 1): Comparison on MMLU (600 questions) and GSM8K (300 questions) against Static-Strong, Static-Weak, Random, RouteLLM-SW (three thresholds), and TS-Cat. RouteLLM-SW comparison uses a different embedding model (noted as a domain-transfer stress test).
3. Multi-model routing (Experiment 2): 5-arm online routing across GPT-4o, Claude-Sonnet-4.5, Qwen-Plus, MiniMax-M1, and DeepSeek-V3—structurally impossible for binary routers without $O(K^2)$ retraining. LinTS achieves stable multi-arm policies across seeds.
4. Ablations: Feature dimensionality, warm-start labels, β sensitivity, and N -arm convergence analysis.

All results are from real API calls recorded in reproducible JSON result files. No simulated data.

2 Related Work

2.1 Supervised LLM Routing

[Ong et al. \[2024\]](#) propose RouteLLM, which trains a classifier on 55K Chatbot Arena human preference labels. Their Similarity-Weighted (SW) router retrieves nearest neighbors from the arena embedding space and uses Elo-weighted win rates as routing scores. While effective when pretraining data aligns with deployment distribution, SW requires retraining when model capabilities change and is architecturally binary. Not Diamond [Contributors \[2024\]](#) uses a random forest (RoRF) over benchmark performance features, also requiring pretraining labels per model pair.

Chen et al. [2023] propose FrugalGPT, a cascade system that queries a sequence of models in increasing cost order, stopping when response quality exceeds a threshold. Dekoninck et al. [2025] provide a unified framework for routing and cascading with theoretical guarantees. These methods avoid per-query pretraining but require quality estimators and cascade thresholds.

Shnitzer et al. [2023] study mixture-of-experts routing for LLMs using benchmark-based correctness predictors. Lu et al. [2023] use reward models to score candidate outputs, requiring multi-model inference. LLM-Blender Jiang et al. [2023] ensembles outputs via pairwise ranking. Šakota et al. [2024] frame model selection as meta-modeling. All learn static decision functions offline.

2.2 Online Learning for LLM Serving

LinUCB Li et al. [2010] is the canonical contextual bandit algorithm for recommendation and routing. Russo et al. [2018] provide a tutorial on Thompson Sampling, showing it matches LinUCB’s regret while requiring less tuning. Abeillé and Lazaric [2017] prove $O(d\sqrt{T\log T})$ regret for Linear Thompson Sampling. NeuralUCB Zhou et al. [2020] extends LinUCB to nonlinear reward models using deep networks with UCB exploration on learned representations.

2.3 Comparison

Table 1: RouteSmith vs. prior work across four dimensions: online learning, K -arm support, zero-pretraining, and interpretability.

Method	Online	K -arm	No labels	Interpretable
RouteLLM-SW Ong et al. [2024]	×	×	×	×
Not Diamond (RoRF) Contributors [2024]	×	×	×	×
FrugalGPT Chen et al. [2023]	✓	✓	×	×
Dekoninck et al. (cascade) Dekoninck et al. [2025]	✓	✓	×	×
TS-Cat	✓	✓	✓	×
LinUCB-27d (Ours)	✓	✓	✓	✓
LinTS-27d (Ours)	✓	✓	✓	✓

3 Method

3.1 Problem Formulation

We model LLM routing as a contextual bandit problem. At each time step t :

1. A query $x_t \in \mathcal{X}$ arrives (e.g., a natural language prompt).
2. The router selects a model arm $a_t \in \mathcal{A} = \{1, \dots, K\}$.
3. The router receives a reward $r_t = R(a_t, x_t) \in [0, 1]$.

The reward function balances quality and cost:

$$R_t = \alpha_q \cdot \text{acc}(a_t, x_t) - \alpha_c \cdot \frac{c_{a_t}}{c_{\max}} \quad (1)$$

where $\alpha_q + \alpha_c = 1$, c_{a_t} is the per-query cost of arm a_t , and $c_{\max}$ is the cost of the most expensive arm. For Experiment 1 (binary routing), $\alpha_q = 0.85$, $\alpha_c = 0.15$. For Experiment 2 (quality-dominant), $\alpha_q = 0.85$.

3.2 Feature Space

We extract a 27-dimensional context vector $\phi(x_t) \in \mathbb{R}^{27}$ from each query:

- Message features [0–10]: Log character count, log word count, sentence count, question mark count, number count, long-query indicator, code block pairs, has-question flag, capitalization flag, vocabulary richness, word saturation.
- Extended features [11–16]: Math/reasoning/code/creative keyword scores, difficulty estimate, vocabulary richness (alias).
- Model features [17–26]: Per-model metadata from the registry: cost rates (input/output), quality prior, median latency, log context window size, binary capability flags (function calling, vision, JSON mode), plus zero-padded reserved dimensions for future model-specific embeddings.

All features are L2-normalized before input: $\tilde{\phi}(x_t) = \phi(x_t)/\|\phi(x_t)\|$.

3.3 LinUCB-27d

LinUCB [Li et al. \[2010\]](#) maintains per-arm ridge regression models. For arm a :

$$A_a \leftarrow A_a + \tilde{\phi}(x_t)\tilde{\phi}(x_t)^\top, \quad b_a \leftarrow b_a + r_t \cdot \tilde{\phi}(x_t) \quad (2)$$

$$\hat{\theta}_a = A_a^{-1}b_a, \quad p_a(x_t) = \hat{\theta}_a^\top \tilde{\phi}(x_t) + \beta \sqrt{\tilde{\phi}(x_t)^\top A_a^{-1} \tilde{\phi}(x_t)} \quad (3)$$

The UCB score $p_a(x_t)$ trades off estimated quality (first term) with uncertainty (second term, controlled by β). Updates use Sherman-Morrison rank-1 matrix updates, requiring $O(d^2) = O(729)$ operations per step with no matrix inversion.

3.4 LinTS-27d (Adapted from Abeillé & Lazaric, 2017)

We adapt Linear Thompson Sampling [Abeillé and Lazaric \[2017\]](#) for LLM routing. For each arm a , LinTS maintains a Gaussian posterior over the weight vector θ_a :

$$p(\theta_a \mid \mathcal{D}_a) = \mathcal{N}(\mu_a, v^2 \Sigma_a) \quad (4)$$

where $\Sigma_a = A_a^{-1}$, $\mu_a = \Sigma_a b_a$. At each step, we sample:

$$\tilde{\theta}_a \sim \mathcal{N}(\mu_a, v^2 \Sigma_a) \quad \forall a \in \mathcal{A} \quad (5)$$

and select $a^* = \arg \max_a \tilde{\phi}(x_t)^\top \tilde{\theta}_a$.

Key advantage over LinUCB: LinTS has no β hyperparameter. Exploration is proportional to posterior variance, which decays naturally as data accumulates. LinTS achieves $O(d\sqrt{T \log T})$ cumulative regret [Abeillé and Lazaric \[2017\]](#), matching LinUCB’s order with zero manual tuning.

3.5 Relation to Full RouteSmith System

This paper focuses on RouteSmith’s online contextual bandit router. The full RouteSmith product [Contributors \[2025\]](#) includes additional components not evaluated here:

- Random forest quality predictor: A supervised router using RoRF-style features with cold-start $\rightarrow$ warm-up $\rightarrow$ learned phase progression. Serves as the default strategy when offline calibration data is available.

- Semantic cache: Embedding-based response caching for semantically similar queries.
- Feedback collection: Production telemetry for continuous routing improvement.
- Multi-strategy architecture: Cascade, parallel, and provisioned-first routing strategies for diverse deployment requirements.

The LinTS bandit router is one of multiple routing strategies available in RouteSmith, suitable for zero-shot deployment scenarios.

3.6 TS-Cat Baseline

As a context-free bandit baseline, TS-Cat maintains a $\text{Beta}(\alpha_k, \beta_k)$ posterior per query category k :

$$\theta_k \sim \text{Beta}(\alpha_k, \beta_k), \quad \alpha_k, \beta_k \leftarrow 1.0 \text{ (uniform prior)} \quad (6)$$

Route to strong if $\theta_k > 0.5$. TS-Cat captures category-level routing preferences but ignores query-level features.

4 Experiments

4.1 Experiment 1: Binary Routing Comparison

4.1.1 Setup

We evaluate all methods on 600 MMLU questions (5 categories $\times$ 120) and 300 GSM8K math word problems. Models: strong = openai/gpt-4o, weak = openai/gpt-4o-mini (via OpenRouter). Bandit methods run with 5 independent random seeds; mean $\pm$ 95% CI reported. RouteLLM-SW is deterministic per threshold—point estimate with embedding fallback noted. We do not perform formal significance tests (e.g., paired bootstrap, McNemar) given the limited number of seeds; CIs are reported for descriptive purposes only.

Reward: $R_t = 0.85 \cdot \text{acc}(a_t, x_t) - 0.15 \cdot c_{a_t}/c_{\max}$.

4.1.2 Results

Discussion. LinUCB vs LinTS. LinUCB-27d achieves the highest MMLU accuracy (78.2%) and APGR (1.126), exceeding Static-Strong by routing selectively—choosing strong-arm on queries where cost-quality reward favors it. LinTS-27d matches Random Router on MMLU (75.8%, APGR=0.593) while achieving higher accuracy on GSM8K (92.5% vs 94.0% random), reflecting better uncertainty quantification on math problems. The key distinction: LinUCB requires β tuning (APGR varies 0.407–0.741 across $\beta \in \{0.5, 1.5, 3.0\}$, see §5), while LinTS requires no hyperparameter.

RouteLLM-SW (domain-transfer stress test). RouteLLM-SW routes 0% of queries to strong across all thresholds. Win-rate scores cluster at 0.218–0.233, below all thresholds. This reflects domain mismatch between Chatbot Arena conversational embeddings and structured MMLU/GSM8K benchmarks, exacerbated by the degraded embedding model (all-MiniLM-L6-v2 384-dim vs. original text-embedding-3-small). We emphasize this comparison uses a different embedding model and should not be interpreted as a claim about RouteLLM-SW’s native performance. Rather, it demonstrates that online bandits learn task-specific routing without relying on pre-calibrated domain embeddings.

Table 2: Experiment 1: Binary routing comparison on MMLU (600 questions) and GSM8K (300 questions). All results from real API calls via OpenRouter. Note: RouteLLM-SW evaluation uses all-MiniLM-L6-v2 fallback embeddings (original text-embedding-3-small not available via OpenRouter); results reflect domain transfer, not native performance.

Method	MMLU Acc	MMLU Cost	GSM8K Acc	GSM8K Cost	APGR
Static-Strong (GPT-4o)	77.7%	\$0.214	97.3%	\$0.904	1.000
Static-Weak (GPT-4o-mini)	73.2%	\$0.012	91.7%	\$0.060	0.000
Random Router	75.8%	\$0.114	94.0%	\$0.519	0.593
RouteLLM-SW ($\tau=0.3$)	72.7%	\$0.012	92.3%	\$0.060	-0.111
RouteLLM-SW ($\tau=0.5$)	72.7%	\$0.012	92.0%	\$0.060	-0.111
RouteLLM-SW ($\tau=0.7$)	72.2%	\$0.012	90.3%	\$0.060	-0.222
TS-Cat (5 seeds)	72.4 $\pm$ 0.4%	\$0.053	91.1 $\pm$ 0.6%	\$0.067	-0.163
LinUCB-27d (5 seeds)	78.2 $\pm$ 0.2%	\$0.196	91.7 $\pm$ 0.5%	\$0.126	1.126
LinTS-27d (5 seeds)	75.8 $\pm$ 0.4%	\$0.116	92.5 $\pm$ 0.4%	\$0.195	0.593

TS-Cat. TS-Cat performs below Static-Weak on MMLU (APGR= -0.163) because the per-category binary bandit converges slowly when categories are heterogeneous—it cannot leverage query-level features.

4.2 Experiment 2: Multi-Model Quality Routing

4.2.1 Setup

Five model arms via OpenRouter: GPT-4o (\$2.50/\$10.00 per 1M), Claude-Sonnet-4.5 (\$3.00/\$15.00), Qwen-Plus (\$0.40/\$1.20), MiniMax-M1 (\$0.30/\$1.10), DeepSeek-V3 (\$0.20/\$0.77 input/output). Reward: $R_t = 0.85 \cdot \text{acc}(a_t, x_t) - 0.15 \cdot c_{a_t}/c_{\max}$. Evaluated on 600 MMLU questions with 3 random seeds (42, 43, 44).

RouteLLM-SW is not applicable to this setting: it is architecturally binary (strong/weak) and extending to $K > 2$ arms would require $O(K^2)$ separate classifiers.

4.2.2 Results

Table 3: Experiment 2: LinTS-5arm multi-model routing on MMLU (600 questions, 3 seeds). Routing distribution shows fraction of queries sent to each model arm.

Seed	Accuracy	Cost (USD)	Routing Distribution
42	71.0%	\$0.104	GPT-4o 34%, Qwen-Plus 46%, DeepSeek-V3 10%, Claude 4%, MiniMax 5%
43	70.7%	\$0.131	GPT-4o 45%, Qwen-Plus 32%, DeepSeek-V3 12%, Claude 5%, MiniMax 5%
44	71.3%	\$0.117	GPT-4o 40%, Qwen-Plus 41%, DeepSeek-V3 10%, Claude 4%, MiniMax 5%
Mean	71.0 $\pm$ 0.3%	\$0.117	GPT-4o $\approx$ 40%, Qwen-Plus $\approx$ 40%, DeepSeek-V3 $\approx$ 11%

Discussion. The 5-arm router concentrates routing on GPT-4o ($\approx$ 40%) and Qwen-Plus ($\approx$ 40%), with DeepSeek-V3 ($\approx$ 11%) capturing knowledge-heavy MMLU categories. Claude-Sonnet-4.5 and

MiniMax-M1 each receive $\approx 4\text{--}5\%$. The average accuracy (71.0%) is below Static-Strong (77.7%) because the 5-arm reward function balances cost: at \$0.117 total cost vs \$0.214 for always-strong, the router achieves 45% cost reduction.

The routing distribution is stable across seeds (coefficient of variation $< 10\%$ for each arm), confirming LinTS-27d converges to a consistent routing policy in the multi-arm setting. Routing patterns vary meaningfully by subject category, validating that the 27-dim feature vector provides discriminative routing signal beyond random allocation.

5 Analysis and Ablations

5.1 Feature Dimensionality

We compare LinTS variants using 11, 17, and 27 feature dimensions (zero-padding to 27d). Results on MMLU (seed=42): 11d achieves APGR=0.778, 17d achieves APGR=0.630, and 27d achieves APGR=0.704.

The non-monotonic trend (11d $>$ 27d $>$ 17d) is unexpected and warrants cautious interpretation. One hypothesis is that the 6 extended keyword features (indices 11–16) introduce noise for MMLU’s multiple-choice format, while the zero-padded model-feature dimensions (17–26) in the 27d variant reduce effective weight of noisy features, partially recovering performance. However, alternative explanations are equally plausible: the single seed may be insufficient for stable estimates, L2-normalization may interact with dimensionality in confounding ways, or the feature space may benefit from more deliberate engineering. We report this as an exploratory result and recommend multi-seed validation before drawing conclusions about optimal feature dimensionality.

5.2 Warm-Start Labels

We initialize LinTS with 0, 100, or 500 oracle labels (always reward arm 1, the strong model) before online evaluation on the remaining 100 queries. Using subset-corrected baselines (strong/weak accuracy recomputed on the same 100 eval queries to avoid data leakage), APGR improves from 0.759 (cold start) to 0.897 (100 labels) and 0.800 (500 labels). The non-monotonic improvement—100 labels outperforms 500—arises because 500 oracle labels strongly bias the posterior toward the strong arm, reducing exploration on the remaining 100 eval queries. Even 100 warm-start labels substantially reduce cold-start regret, suggesting that a small labeled calibration set is beneficial when available.

5.3 LinUCB β Sensitivity vs LinTS

LinUCB APGR at $\beta \in \{0.5, 1.5, 3.0\}$: 0.407, 0.667, 0.741 respectively. Higher β encourages more exploration, routing more queries to the strong arm and increasing APGR at the cost of higher API spend. LinTS (no β) achieves APGR=0.519, falling between $\beta = 0.5$ and $\beta = 1.5$ without any hyperparameter tuning. This confirms LinTS provides parameter-free uncertainty quantification via posterior sampling, making it preferable in deployment settings where calibration is impractical.

5.4 Distribution Shift

LinTS-27d trained and evaluated on GSM8K achieves $92.5\% \pm 0.4\%$ (APGR=0.153 using GSM8K baselines: strong=97.3%, weak=91.7%). This is notably higher than TS-Cat (91.1%, APGR=−0.106),

showing that the 27-dim feature representation provides meaningful routing signal on math problems beyond simple category-level preferences. The lower APGR on GSM8K relative to MMLU reflects the harder problem: GSM8K’s strong/weak accuracy gap is only 5.6 pp (vs. 4.5 on MMLU).

5.5 Reproducibility

All results in this paper are from real API calls with fixed random seeds, ensuring full reproducibility. Experiment scripts, configurations, and random seeds are provided in the supplementary material. All hyperparameters are stored in the result files.

6 Routing Latency

RouteSmith’s routing decision—feature extraction, bandit inference, and arm selection—adds negligible overhead to LLM inference. Table 4 reports micro-benchmarks on a MacBook Pro M3 Pro (single core, Python 3.13), averaged over 1,000 trials.

Table 4: Routing latency breakdown (1,000 trials, single core).

Component	Mean (ms)	P99 (ms)
Feature extraction (27d)	0.12	0.28
LinUCB inference ($K=2$)	0.03	0.06
LinUCB inference ($K=5$)	0.07	0.13
LinTS inference ($K=2$)	0.05	0.10
LinTS inference ($K=5$)	0.11	0.21
Total (LinTS, $K=5$)	0.23	0.49

Total routing overhead is under 0.5ms P99 for a 5-arm deployment—well within typical LLM inference latencies of 500–5,000ms. The dominant cost is feature extraction (string tokenization and regex); bandit inference scales linearly with K and remains sub-millisecond for practical model pool sizes.

7 Conclusion

We presented the online contextual bandit component of RouteSmith, a multi-strategy LLM routing system. LinTS-27d, adapted from Linear Thompson Sampling [Abeillé and Lazaric \[2017\]](#), achieves competitive routing without pretraining data, scales to K arms naturally, and eliminates the β hyperparameter that LinUCB requires. The full RouteSmith product additionally includes a random forest quality predictor and semantic caching for deployment scenarios where offline calibration data is available.

Structural advantages of online bandit routing. Compared to static supervised routers, online bandit methods (1) require zero pretraining labels before routing queries, (2) extend naturally to $K > 2$ arms without $O(K^2)$ retraining, (3) continue learning from production feedback, and (4) provide per-feature interpretability through learned weight vectors. These properties make bandit routing attractive for zero-shot multi-model deployments, though supervised routers remain strong options when pretraining data aligns with the target distribution.

Limitations.

- Evaluation scope: Our experiments use 600 MMLU questions (4.3% of the full 14,042-question benchmark), 300 GSM8K problems (22.7% of 1,319), and 100 MBPP coding problems. No evaluation covers conversational tasks, code generation, or real-world assistant workloads.
- Statistical significance: We report $\pm 95\%$ CIs from 5 seeds descriptively but do not perform formal significance tests (paired bootstrap, McNemar). Where CIs overlap (e.g., LinTS at 75.8% vs. Random at 75.8% on MMLU), the methods are not distinguishable at 5 seeds.
- RouteLLM-SW comparison: We use all-MiniLM-L6-v2 (384-dim) fallback embeddings rather than the original text-embedding-3-small. This degraded baseline prevents drawing strong comparative conclusions about RouteLLM’s native performance.
- No cascade comparison: We do not benchmark against FrugalGPT or Dekoninck et al.’s cascade-based approach with confidence thresholds. These are important baselines for future work.
- 27d vs. 35d gap: The benchmark uses 27-dimensional features while the production LinTS Predictor defaults to 35 dimensions. Results may not transfer directly to the production configuration.
- Non-stationarity: LinTS’s regret bound assumes stochastic linear bandits; real deployments face non-stationarity from model updates and shifting query distributions.
- Scale: Number of arms tested is 5—scaling behavior beyond this is unknown. All experiments use OpenRouter which adds latency/error variance compared to direct API access.

Future work. NeuralUCB (implemented but not yet benchmarked due to insufficient queries for stable neural training), comparisons against Not Diamond’s RoRF and Dekoninck et al.’s cascade framework, scaling to full benchmark sizes, evaluation on conversational and code-generation tasks, multi-turn conversation routing, and production deployment with real user traffic.

References

- M. Abeillé and A. Lazaric. Linear thompson sampling revisited. *Electronic Journal of Statistics*, 11(1):2499–2541, 2017.
- P. Aggarwal, A. Madaan, Y. Yang, and Mausam. Automix: Automatically mixing language models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024.
- L. Chen, M. Zaharia, and J. Zou. Frugalgpt: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*, 2023.
- N. D. Contributors. Not diamond: Pareto-optimal llm routing. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2024. Open-source random forest router (RoRF).
- R. Contributors. Routesmith: Adaptive llm execution engine, 2025. Open-source project. <https://github.com/yunpengl9071/routesmith>.
- J. Dekoninck et al. A unified approach to routing and cascading for llms. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2025.

- D. Jiang, X. Ren, and B. Y. Lin. Llm-blender: Ensembling large language models with pairwise ranking and generative fusion. In Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics, 2023.
- L. Li, W. Chu, J. Langford, and R. E. Schapire. A contextual-bandit approach to personalized news article recommendation. In Proceedings of the 19th International Conference on World Wide Web, pages 661–670, 2010.
- K. Lu, H. Yuan, R. Lin, J. Lin, Z. Yuan, C. Zhou, and J. Zhou. Routing to the expert: Efficient reward-guided ensemble of large language models. arXiv preprint arXiv:2311.08692, 2023.
- I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kadous, and I. Stoica. Routellm: Learning to route llms with preference data. In Proceedings of the International Conference on Machine Learning (ICML), 2024.
- D. J. Russo, B. Van Roy, A. Kazerouni, I. Osband, and Z. Wen. A tutorial on thompson sampling. Foundations and Trends® in Machine Learning, 11(1):1–96, 2018.
- M. Šakota, M. Peyrard, and R. West. Fly-swat or cannon? cost-effective language model choice via meta-modeling. arXiv preprint arXiv:2308.06077, 2024.
- T. Shnitzer, A. Ou, M. Silva, K. Soule, Y. Sun, J. Solomon, N. Thompson, and M. Yurochkin. Large language model routing with benchmark datasets. arXiv preprint arXiv:2309.15789, 2023.
- D. Zhou, L. Li, and Q. Gu. Neural contextual bandits with ucb-based exploration. In International Conference on Machine Learning, pages 11492–11502, 2020.